



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Reconocimiento de Lenguaje
de Signos ASL mediante IA:
Implementación y
Comparativa de Rendimiento
en Raspberry Pi.
Documentación Técnica**



Presentado por Hugo Gómez Martín
en Universidad de Burgos — 7 de julio de 2025

Tutor: Fernando Rivas Navazo

Tutor: Daniel Urda Muñoz

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	5
Apéndice B Especificación de Requisitos	11
B.1. Introducción	11
B.2. Objetivos generales	11
B.3. Catálogo de requisitos	12
B.4. Especificación de requisitos	14
Apéndice C Especificación de diseño	25
C.1. Introducción	25
C.2. Diseño de datos	25
C.3. Diseño procedimental	26
C.4. Diseño arquitectónico	30
Apéndice D Documentación técnica de programación	35
D.1. Introducción	35
D.2. Estructura de directorios	35
D.3. Manual del programador	37

D.4. Compilación, instalación y ejecución del proyecto	39
D.5. Pruebas del sistema	40
Apéndice E Documentación de usuario	41
E.1. Introducción	41
E.2. Requisitos de usuarios	41
E.3. Instalación	42
E.4. Manual del usuario	42
Apéndice F Anexo de sostenibilización curricular	53
F.1. Introducción	53
F.2. Competencias de sostenibilidad	53
F.3. Conclusión	54
Bibliografía	55

Índice de figuras

C.1. Diagrama de flujo de Detección en tiempo Real	27
C.2. Diagrama de flujo de Clasificación de imagen estática	28
C.3. Diagrama de flujo de Entrenamientos de modleos personalizados	29
C.4. Esquema de la arquitectura MVC [3]	30
C.5. Logotipo de la aplicación ASL Detection	33
C.6. Paleta de colores implementada en la aplicación	33
C.7. Captura de la interfaz tiempo real	34
D.1. Captura de las URL que nos muestra Streamlit por consola . . .	40
E.1. Captura de la pestaña de nuestra web con logo y título	43
E.2. Captura de la interfaz de inicio de la aplicación	43
E.3. Captura ampliada de la interfaz de inicio para apreciar mejor la descripción	44
E.4. Captura de la flecha establecida en la barra lateral	44
E.5. Captura del menú presente en la barra lateral	45
E.6. Captura de la interfaz para elegir un modelo (FP32 o FP16) . .	46
E.7. Captura de la interfaz inicial de Tiempo Real	46
E.8. Captura de la parte de inferencia de la interfaz Tiempo Real . .	47
E.9. Captura de la parte de métricas de la interfaz tiempo real . . .	48
E.10. Captura de la interfaz inicial de Foto	48
E.11. Captura de la interfaz inicial foto que no acepta el fichero por incompatibilidad	49
E.12. Captura de la interfaz foto una vez realizada la inferencia	49
E.13. Captura de la interfaz de entrenamiento inicial	50
E.14. Captura de la interfaz entrenamiento añadiendo 1000 imágenes .	50
E.15. Captura de la interfaz de entrenamiento en el proceso de entre- namiento	51

E.16. Captura de la interfaz de entrenamiento finalizada	52
E.17. Captura del fichero .zip descomprimido	52

Índice de tablas

A.1. Coste anual total.	8
A.2. Librerías, versiones y licencias establecidas en el proyecto.	8
B.1. CU-1 Detección en tiempo real.	15
B.2. CU-2 Clasificar imagen estática.	16
B.3. CU-3 Entrenamiento de modelos personalizados.	17
B.4. CU-4 Cargar imágenes de entrenamiento.	18
B.5. CU-5 Configurar hiperparámetros.	19
B.6. CU-6 Visualización del progreso del entrenamiento.	20
B.7. CU-7 Descargar modelo entrenado.	20
B.8. CU-8 Seleccionar modelo de inferencia.	21
B.9. CU-9 Navegar entre secciones.	21
B.10.CU-10 Visualizar métricas del sistema.	22
B.11.CU-11 Limpieza de datos temporales.	23

Apéndice A

Plan de Proyecto Software

A.1. Introducción

En este capítulo se detallarán aspectos fundamentales del proyectos como son la planificación temporal del proyecto y su receptiva viabilidad tanto económica como legal.

La planificación temporal se establecerá un listado/cronograma de entregas con un breve resumen en cada una hasta la finalización del proyecto.

El estudio de la viabilidad se desglosará en dos capítulos: **viabilidad económica** donde se estimarán y detallan distintos costes asociados al proyecto y la **viabilidad legal**, donde se examinan distintas normativas, leyes aplicables a licencias de software o distintas leyes aplicables.

A.2. Planificación temporal

El proyecto se ha ido planificando progresivamente mediante distintas entregas/sprints realizados durante prácticamente 5 meses. Cada entrega abarcó tareas concretas como análisis, implementación y prueba, progresando así desde la configuración inicial hasta la entrega del proyecto.

Todas estas entregas fueron consultadas con los tutores para establecer distintas funcionalidades y concretar fechas de cierre, aunque es cierto que estas entregas podrían alargarse debido al horario estricto y así poder concretar exitosamente estas funcionalidades.

A continuación se muestra la planificación temporal:

Entregable inicial (9/12/2024 - 26/12/2025)

En esta primer a entrega se estableció la principal hoja de ruta del proyecto junto con los diferentes objetivos principales a implementar. Se decidió desarrollar inicialmente como punto de partida todo lo relacionado con la generación del conjunto de datos, desde su respectivo *script*, hasta el guardado de este. Se corrigieron aspectos como cierres inesperados y la aplicación de un menú vía terminal para añadir mas clases o imágenes.

Además se decidieron las clases y límites del conjunto de datos, además de implementar en el proyecto la librería *cvzone* para poder detectar la mano y recortar automáticamente la región de interés haciendo un *dataset* generalizado.

Entregable 2 (13/1/2025 - 20/1/2025)

Esta entrega fue sumamente importante, debido a que abordamos en la reunión qué tipo de modelo elegir para poder clasificar el reconocimiento de gestos. Finalmente se decidió (Gracias a la experiencia de los tutores) usar para nuestro proyecto una red neuronal convolucional (CNN).

Desde ese inicio del plazo de esta entrega, se aprendieron fundamentos de redes neuronales convolucionales y *TensorFlow/Keras* para poderlo aplicar correctamente y crear una red neuronal.

Al principio se implementó un esqueleto de esta red neuronal convolucional la cual fuimos actualizando para poder adaptarla al *dataset* y así hacer que convergiera mejor inicialmente sin ningún tipo de optimización.

Entregable 3 (3/2/2024 - 16/2/2025)

Los distintos objetivos de esta entrega fueron: establecer correctamente el clasificador mejorando la salida por pantalla aportándole más información al usuario, y experimentar con dos nuevos conjuntos de datos públicos extraídos de *Kaggle* para ver como se comportaba nuestro modelo cuando agregábamos distintos conjuntos de datos.

Entregable 4 (17/3/2024 - 30/4/2025)

Los objetivos de esta entrega fueron: reducir el tamaño del repositorio eliminando principalmente los modelos exportados, y principalmente, ayudar a exportar nuestro modelo a otras computadoras insertando las librerías

utilizadas en un fichero de texto llamado *requirements* para poder exportarlo de una manera mas fácil y rápida.

Es cierto que nuestra arquitectura del modelo CNN principal en fases iniciales del proyecto no estaba para nada optimizada y algunos modelos exportados llegaban a pesar bastante como para poder alojarlos en el repositorio, por eso se decidió apartarlos momentáneamente hasta poderlos optimizar.

Entregable 5 (7/4/2024 - 20/4/2025)

En esta entrega tuvimos como objetivos: optimizar la arquitectura previa de la CNN principal para poder exportar el modelo de una forma mas ligera sin sacrificar mucho rendimiento, y aplicar *Random Search* para encontrar los mejores hiperparámetros para nuestro modelo, además de implementarlo en *Jupyter Notebook* para poder realizar la búsqueda de hiperparámetros con *Colab*.

Otros objetivos establecidos para este proyecto fueron: implementar las principales gráficas de pérdida/exactitud para ver visualmente la convergencia del modelo y comprobar visualmente el sobreajuste; ampliar el *dataset* con fotos de una segunda persona para implementar nuevas características a las clases además de crear un *script* llamado para desordenar las imágenes para el correcto *split* del modelo. Se añadió la plantilla de documentación en el repositorio principal, pero se documentó totalmente en *overleaf* donde la agregaremos una vez finalizada al repositorio.

Además, Implementamos el modelo exportado de **Teachable Machine** para poder compararlo con nuestro modelo además de tener un modelo auxiliar que, a lo largo del proyecto, nos resultará de gran utilidad y eliminamos el *dataset* experimental después de aumentar de tamaño el nuestro.

Entregable 6 (28/4/2025 - 11/5/2025)

Fueron numerosos objetivos a implementar en este entregable, estos fueron: mejorar la estructura del proyecto, modificar el *Random search* para poder correr en local gracias al descubrimiento de la librería **TensorFlow Metal**, convertir los modelos a dos versiones más optimizadas cuantizadas con **TFLite** (FP32 y FP16)

Es cierto que los objetivos más destacados de este entregable fueron: empezar a implementar la exportación a la web, añadiendo primero el clasificador y ver como se comportaba principalmente en la Raspberry Pi

con una cámara normal conectada vía USB. Además, se empezó a adaptar nuestro modelo para poder ser compatible con la cámara IA de Raspberry Pi cuantizando el modelo a *INT8* además de modificar diferentes capas del modelo debido a su incompatibilidad con el conversor oficial de Sony.

Finalmente, se corrigieron aspectos del código generales y se eliminaron ficheros que ya no se usaban en el modelo (fueron simplemente experimentales a la hora de aprender nuevas técnicas para mejorar el entrenamiento del modelo), como por ejemplo, el conjunto de datos aumentado *augmented dataset* debido a que se descubrió que se podía hacer un *augmentation on-the-fly* es decir, mientras se cargan en el pipeline antes del entrenamiento sin ocupar espacio en el disco duro generando un conjunto de datos nuevo.

Entregable 7 (19/5/2025 - 1/6/2025)

En este entregable se siguieron los siguientes objetivos: reestructurar de una forma clara el proyecto por segunda vez normalizando los nombres de los distintos ficheros Python que componen el proyecto y seguir intentando convertir el modelo convertido a *onnx* para intentar convertir de otra forma al formato que acepta cámara IA de Raspberry pi, pero este intento no produjo los resultados esperados. Por tanto, al no obtener resultados, redirigimos el objetivo inicial de desplegar nuestro modelo en la cámara IA de Raspberry Pi, quedando su implementación inviable por el momento.

Entregable 8 (9/6/2025 - 22/6/2025)

Lo siguiente objetivos de la entrega 8 son: simplificar las distintas rutas y variables globales añadiendo un *script* específico para el manejo de estas (*config.py*), implementación de la web con *Streamlit* inicial con un apartado de clasificador en tiempo real con monitoreo de *hardware* integrado en la interfaz, además de un apartado para la clasificación estática de imágenes y otro para poder entrenarlo en la propia aplicación añadiendo hasta 26 clases sin límite de imágenes donde podemos editar 3 hiperparámetros (*learning rate*, *epochs*, *dropout*). En esta misma interfaz también se puede monitorear los valores de *loss* y *accuracy* en cada época durante el monitoreo. Además, se le realizó mejoras al *frontend* introduciendo colores oficiales de Raspberry, añadiendo un menú principal, un desplegable para seleccionar el modelo optimizado y un pie de pagina.

Además otros objetivos destacables fueron: añadir scripts automatizados para que el usuario solo se tenga que preocupar para instalar la versión de Python requerida y ejecutar directamente el *script .bat* si tu sistema

operativo es Windows o ejecutar el *script* en *bash .sh* si tu sistema está basado en Linux (en nuestro proyecto usamos *pi OS*). En el modelo también establecemos una nueva forma de cargar los datos de forma mas optimizada gracias a *Tensorflow Keras*.

Entregable Final (23/6/2024 - 8/7/2025)

Los objetivos finales fueron: mejora del modelo introduciendo un modelo preentrenado (converge mucho mejor que el anterior realizado desde 0) proporcionado por la librería *Keras* llamado *EfficientNetB0*.

Como objetivo también quisimos alojar en la web nuestra aplicación, pero no fue posible debido a que la librería *OpenCV* a la hora de mostrar la inferencia en tiempo real no funcionaba una vez alojada. por lo que tuvimos que redirigir ese objetivo a realizar un ejecutable (*.exe*) para que al menos el usuario pueda ejecutar la aplicación sin depender de dependencias, únicamente de tener Python 3.11 o superior instalado. Además, se creó un logo para mostrar en la web para darla identidad ademas de agregar este logo a la pestaña y al ejecutable.

Como últimos objetivos fueron comentar todo el código del proyecto, finalizar a tiempo la memoria y anexos, y descartar fichero que se habían quedado obsoletos como los *Jupyter Notebooks*.

Echando la vista atrás, es cierto que podemos apreciar un cierto desbalance en la carga de trabajo en los diferentes *sprints*, justificado por la necesidad de adaptar las estas diferentes tareas a mi disponibilidad horaria, que ha sido limitada.

A.3. Estudio de viabilidad

En este capitulo de detallará el estudio sobre la viabilidad económica y legal del proyecto.

Viabilidad económica

es importante revisar la viabilidad económica del proyecto para saber una aproximación de los gastos y beneficios que nos otorgará el proyecto.

Costes

Los costes se calcularán de forma anual con una amortización total de unos 5 años.

costes de personal

Supongamos desde un punto de vista empresarial, que hemos sido contratados por una entidad o empresa durante 5 meses, unas para la realización de este proyecto.

Además, contemos con que el programador es junior, por lo tanto, partamos del que su sueldo anual es de 25000€ [1]. Se establecen 14 pagas: $25000/14 = 1785,71\text{€}$, reduciéndolo a media jornada: $1785,71/2 = 892,85\text{€}$ mensuales. Si el contrato ha sido anual, el coste del programador junior será de: $892,85 \times 12 = 10714,2\text{€}$ brutos.

Por lo tanto, el coste de personal para la empresa sería de unos: 10714,2€ brutos.

Costes de hardware

Se ha empleado el siguiente *hardware* en la realización del proyecto:

- Ordenador Portatil ASUS ZenBook 13: 850€
- Raspberry Pi 5: 100€
- Mac Pro 6.1: 500€
- Teclado + Ratón inalámbrico: 30€
- Cámara Play Station 3 Eye: 10€

Este es el coste sobre el precio actual de los productos en el año 2025. Se supone una amortización completa de 5 años, por lo que que coste anual para los distintos productos sería:

- Ordenador Portatil ASUS ZenBook 13: $850/5 = 170\text{€}$
- Raspberry Pi 5: $100/5 = 20\text{€}$
- Mac Pro 6.1: $500/5 = 100\text{€}$
- Teclado + Ratón inalámbrico: $30/5 = 6\text{€}$

- Cámara Play Station 3 Eye: $10/5 = 2\text{€}$

Por lo tanto, el coste anual del hardware ascendería a 298€

Coste de software

El único coste de software de pago utilizado en este proyecto ha sido el sistema operativo desarrollado por Microsoft:

- Windows 10 Pro 64 bits: 130€

Se sigue suponiendo una amortización de 5 años, por lo que el coste anual para el software sería:

- Windows 10 Pro 64 bits: $200/5 = 40\text{€}$

Por lo tanto, el coste anual del software sería: 40€

Otros costes

Además, se tienen que tener en cuenta los distintos gastos que se deben realizar para la entrega del proyecto:

- 3 Pendrives para la entrega: 30€
- encuadernación de la memoria: 15€

Estos gastos están supuestos a amortizar en un año, por lo que su coste total en un año será de 45€

con todos estos costes calculados, el coste anual es:

Coste	Total
Personal	10714,2€
Hardware	298€
Software	40€
Otros	45€
Total	11097,2€

Coste	Total
-------	-------

Tabla A.1: Coste anual total.

Viabilidad legal

En este capítulo se realizará la investigación las diferentes licencias que establecen las diferentes herramientas y librerías que hemos utilizado durante el proyecto exponiendo su versión y tipo de licencia.

Librería	Versión	Licencia
cvzone	1.5.6	MIT
opencv-python	4.10.0.84	Apache License 2.0
numpy	1.24.3	BSD 3-Clause License
tensorflow	2.13.1	Apache License 2.0
tensorflow-metal	0.8.0	MIT
mediapipe	0.10.18	Apache License 2.0
keras-tuner	1.4.7	Apache License 2.0
matplotlib	3.9.4	Python Software Foundation License
psutil	7.0.0	BSD-3-Clause
streamlit	1.45.1	Apache License 2.0
streamlit-option-menu	0.4.0	MIT License
pandas	2.3.0	BSD 3-Clause License
keras	2.13.1	Apache License 2.0
scikit-learn	1.6.0	BSD 3-Clause License

Tabla A.2: Librerías, versiones y licencias establecidas en el proyecto.

Todas estas librerías que han sido empleadas en el proyecto se publican bajo licencias de Software libre como MIT, APACHE 2.0 y BSD 3-Clause, por lo que su uso no dispone trabas adicionales al código que hemos desarrollado.

En particular, la licencia MIR permite utilizar, modificar y distribuir el software siempre que se conserve el aviso de copyright y se cite a los

autores. Por lo tanto se nos concede esa libertad de publicar nuestro proyecto con la licencia que consideremos adecuada, bastando con incluir menciones requeridas a los creadores de estas librerías que hemos integrado.

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

En este capítulo se detallan todos los Objetivos generales que recogimos al inicio del proyecto para seguir una hoja de ruta en la realización del proyecto además de detallar los requisitos que tiene que cumplir la aplicación.

B.2. Objetivos generales

Los objetivos generales que se establecieron al inicio fueron:

- Desarrollar un sistema que fuese capaz de traducir cada letra de las 26 que componen el abecedario del lenguaje de signos ASL a texto, centrado en el alfabeto manual, que funcione de manera autónoma y ayude a disminuir la barrera de comunicación entre personas sordas o con trastornos de audición y oyentes.
- Demostrar la viabilidad de aplicar técnicas de visión por computador y *deep learning* además de poder optimizarlas en entornos embebidos para el reconocimiento de gestos, alcanzando un rendimiento suficiente para su uso interactivo.
- Comparar el rendimiento de distintas configuraciones y modelos dentro del proyecto, en particular evaluar mejoras obtenidas entre distintos modelos, además de evaluar el rendimiento con la respectiva optimización del modelo.

B.3. Catálogo de requisitos

Una vez hemos interiorizado y comprendido los objetivos del proyecto, tenemos que enumerar los requisitos funcionales y no funcionales de todo el proyecto.

Requisitos funcionales

Son requisitos que definen que acciones debe poder llevar a cabo con éxito en nuestra aplicación:

- **RF-1 Detección en tiempo real:** La aplicación debe reconocer signos de lenguaje ASL directamente desde la cámara del dispositivo.
 - **RF-1.1 Capturar Vídeo:** Permitir al usuario poder capturar video con la cámara de su dispositivo.
 - **RF-1.2 Iniciar y detener captura de vídeo:** Permitir al usuario iniciar y detener la captura con un botón.
 - **RF-1.3 Ejecutar inferencia:** La aplicación debe ejecutar la inferencia fotograma a fotograma con un modelo **TFLite** precargado y superponer en pantalla la etiqueta del gesto y su confianza.
 - **RF-1.4 Mostrar FPS:** Permitir al usuario visualizar el número de FPS en todo momento durante la inferencia.
 - **RF-1.5 Mostrar métricas:** Permitir al usuario visualizar en tiempo real métricas de CPU, RAM y temperatura y una gráfica histórica.
- **RF-2 Clasificar una imagen estática:** La aplicación debe ser capaz de clasificar una imagen aportada por el usuario.
 - **RF-2.1 Admitir formatos generales:** La aplicación debe ser capaz de admitir formatos *.jpg*, *.jpeg* y *.png*.
 - **RF-2.2 Preprocesar la imagen:** La aplicación debe ser capaz de pre-procesar la imagen (*resize* 224×224 px, normalización) y obtener la predicción.
 - **RF-2.3 Mostrar resultados:** Permitir al usuario visualizar la deducción y su porcentaje de confianza, además de la previsualización de la imagen y una barra de progreso.
 - **RF-2.4 Ofrecer descarga:** Permitir al usuario descargar su imagen.

- **RF-3 Entrenamiento de modelos personalizados:** Permitir al usuario entrenar su propio modelo desde la web.
 - **RF-3.1 Definir número de clases:** Permitir al usuario poder definir la cantidad de clases que necesite para clasificar en un rango entre 2 y 26 y poder subir múltiples imágenes por clase y añadir un nombre a cada clase.
 - **RF-3.2 Configurar hiperparámetros:** Permitir al usuario configurar hiperparámetros como *epochs*, *batch size* y *learning rate* dentro de unos rangos controlados por tres *sliders*.
 - **RF-3.3 Entrenar un modelo CNN:** Permitir al usuario poder entrenar el modelo con su respectivo conjunto de datos.
 - **RF-3.4 Mostrar progreso:** Permitir al usuario saber en todo momento a través de una barra de progreso y gráfica en vivo de *train/val accuracy*.
 - **RF-3.5 Permitir descarga:** Permitir al usuario descargar un comprimido (*.zip*) que contiene su modelo sin optimizar (*.h5*), un modelo ya optimizado a *TFlite* y las etiquetas en un fichero de texto.
- **RF-4 Selección y gestión de modelos:** Permitir al usuario poder elegir el modelo de inferencia (FP16/FP32) desde la barra lateral
- **RF-5 Interfaz y navegación:** La aplicación debe ofrecer tres secciones disponibles desde un *sidebar*: *Tiempo real*, *foto* y *entrenamiento*.
 - **RF-5.1 Tema frontend común:** Aplicar un tema común al frontend en toda la interfaz.
- **RF-6 Gestión de datos temporales:** La aplicación debe direccionar las imágenes subidas por el usuario a un directorio temporal y eliminarlas al finalizar el proceso.

Requisitos no funcionales

- **RNF-1 Usabilidad:** La aplicación debe establecer una interfaz de usuario clara (con distintos iconos, textos claros) y intuitiva para proporcionar una navegación sencilla y rápida. Además, todas las funcionalidades tienen que ser completadas en menos de tres clics.

- **RNF-2 Rendimiento:** La aplicación debe permitir una latencia en la inferencia lo suficientemente baja como para poder detectar las letras claramente con cierta fluidez.
- **RNF-3 Portabilidad:** La aplicación debe ser ejecutable sin depender o depender mínimamente de dependencias (tener Python 3.11 o superior instalado) a demás de poder ejecutarlo tanto en equipos Windows/Linux.
- **RNF-4 Escalabilidad:** La aplicación debe poder recibir soporte para añadir nuevas clases o modelos sin modificar el núcleo de la aplicación.
- **RNF-5 Fiabilidad:** La aplicación debe liberar explícitamente la cámara ante errores e informar al usuario sobre diferentes errores en las respectivas funcionalidades.
- **RNF-6 Seguridad y privacidad:** La aplicación debe ejecutarse localmente por el momento. No se envían imágenes ni vídeo a servidores externos manteniendo así la seguridad del usuario.
- **RNF-7 Mantenibilidad:** La aplicación debe tener un diseño modular con componentes aislados para poder así realizar un mantenimiento mas accesible.
- **RNF-8 Eficiencia:** La aplicación debe otorgar disponibilidad de un modelo cuantizado FP16 para reducir el consumo y temperatura en dispositivos embebidos.

B.4. Especificación de requisitos

CU-1	Detección en tiempo real
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-1, RF-1.1, RF-1.2, RF-1.3, RF-1.4, RF-1.5
Descripción	Reconoce gestos ASL en tiempo real a partir de la cámara, mostrando etiqueta, porcentaje de confianza, FPS y métricas del sistema.
Precondición	El usuario debe encontrarse en la; permiso de cámara concedido; tener una cámara conectada al equipo
Acciones	1. El usuario pulsa Iniciar cámara. 2. EL sistema abre el flujo de vídeo. 3. El usuario empieza a realizar gestos ASL. 4. El sistema clasifica cada imagen gracias al modelo y se superpone la etiqueta + confianza. 5. El sistema muestra FPS y gráficas de CPU/RAM/temperatura. 6. El usuario pulsa Detener cámara
Postcondición	Vista vuelve al estado inicial y cámara liberada.
Excepciones	Cámara no encontrada, se avisa vía Streamlit y aborta.
Importancia	Alta

Tabla B.1: CU-1 Detección en tiempo real.

CU-2	Clasificar imagen estática
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-2, RF-2.1, RF-2.2, RF-2.3, RF-2.4
Descripción	Clasifica una foto subida por el usuario, muestra la predicción y otorga la posibilidad de descargar la imagen.
Precondición	El usuario debe encontrarse en la interfaz de clasificación por foto.
Acciones	1. El usuario hace clic o arrastra una imagen con formato válido. 2. El sistema redimensiona y preprocesa. 3. El sistema realiza la inferencia y se muestra la etiqueta + confianza. 4. El usuario puede pulsar la opción de descargar.
Postcondición	La imagen se libera del guardado temporal
Excepciones	Formato no soportado, se avisa vía Streamlit .
Importancia	Alta

Tabla B.2: CU-2 Clasificar imagen estática.

CU-3	Entrenamiento de modelos personalizados
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-3, RF-3.1, RF-3.2, RF-3.3, RF-3.4, RF-3.5
Descripción	Permite al usuario crear y descargar un modelo CNN entrenado con sus imágenes
Precondición	El usuario debe encontrarse en la interfaz de entrenamiento.
Acciones	1. El usuario puede definir la cantidad de clases que necesite además de poder otorgarlas un nombre 2. El usuario puede añadir imágenes compatibles (<i>.jpg</i>, <i>.jpeg</i>, <i>.png</i>) (CU-4 y CU-5) 3. El usuario pulsa el boton entrenar, se inicia y se muestra el progreso (CU-6) 4. El usuario puede pulsar Descargr el modelo (CU-7)
Postcondición	Modelo descargado en un comprimido (<i>.zip</i>) y se borran los datos temporales.
Excepciones	-
Importancia	Alta

Tabla B.3: CU-3 Entrenamiento de modelos personalizados.

CU-4	Cargar imágenes de entrenamiento
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-3.1, RF-6
Descripción	El usuario sube imágenes compatibles y puede añadir o eliminar clases entre un mínimo de 2 y un máximo de 26.
Precondición	Clases definidas
Acciones	1. El usuario elige el número de clases que necesita dentro de un rango de 2 a 26 2. El usuario inserta las imágenes en cada clase 3. El sistema guarda en una carpeta principal y muestra el nombre de la imagen almacenada junto a una aspa para eliminar esa foto si el usuario así lo desea.
Postcondición	Conjunto de datos actualizado y listo para entrenar
Excepciones	Formatos no válidos o tamaño de imagen excesivo
Importancia	Alta

Tabla B.4: CU-4 Cargar imágenes de entrenamiento.

CU-5	Configurar hiperparámetros
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-3.2
Descripción	Ajusta el número de épocas (<i>epochs</i>), el tamaño del lote (<i>batch size</i>) y la tasa de aprendizaje (<i>learning rate</i>) mediante la implementación de <i>sliders</i>
Precondición	El usuario debe encontrarse en la interfaz de entrenamiento.
Acciones	1. EL usuario puede establecer estos hiperparámetros dentro del rango del <i>slider</i> 2. El sistema guarda los valores para el posterior entrenamiento.
Postcondición	Parámetros definids y listos para aplicarse.
Excepciones	-
Importancia	Media

Tabla B.5: CU-5 Configurar hiperparámetros.

CU-6	Visualización del progreso del entrenamiento
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-3.4
Descripción	Muestra en tiempo real la barra de progreso y la gráfica <i>accuracy/loss</i>
Precondición	Entrenamiento en ejecución
Acciones	1. El usuario observa como su modelo va progresando época a época. 2. El sistema actualiza la barra y gráfica.
Postcondición	Entrenamiento completado
Excepciones	falta de memoria, entrenamiento abortado
Importancia	Media

Tabla B.6: CU-6 Visualización del progreso del entrenamiento.

CU-7	Descargar modelo entrenado
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-3.5
Descripción	Genera y descarga un comprimido (.zip) con <i>.h5</i> , <i>.tflipe</i> y <i>labels.txt</i>
Precondición	Entrenamiento finalizado con éxito
Acciones	1. El usuario pulsa el botón Descargar modelo. 2. El sistema empaqueta los ficheros y se inicia la descarga.
Postcondición	Comprimido disponible en la carpeta descargas del navegador.
Excepciones	-
Importancia	Media-Alta

Tabla B.7: CU-7 Descargar modelo entrenado.

CU-8	Seleccionar modelo de inferencia
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-4
Descripción	Cambiar entre modelo FP32 y FP16 desde el sidebar.
Precondición	Modelos presentes en la carpeta
Acciones	1. El usuario abre el desplegable 2. El usuario elige FP32 o FP16 (mas optimizado) 3. El sistema recarga el modelo elegido
Postcondición	Cambio de modelo activo en los modos de inferencia
Excepciones	-
Importancia	Media

Tabla B.8: CU-8 Seleccionar modelo de inferencia.

CU-9	Navegar entre secciones
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-5, RF-5.1
Descripción	Muestra un menú de 4 elementos: Inicio, Tiempo real, Foto o Entrenamiento, los cuales se pueden elegir desde la barra lateral.
Precondición	Aplicación abierta
Acciones	1. El usuario hace clic en la opción deseada 2. El sistema cambia la interfaz
Postcondición	Nueva interfaz visible y operativa
Excepciones	-
Importancia	Alta

Tabla B.9: CU-9 Navegar entre secciones.

CU-10	Visualizar métricas del sistema
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-1.5
Descripción	Representa valores y gráficas por pantalla de las métricas en tiempo real de CPU, RAM y temperatura durante la detección en tiempo real.
Precondición	CU-1 en ejecución
Acciones	1. El usuario observa los valores de la gráfica pudiendo descargarla en foto o ampliarla si lo desea. 2. El sistema actualiza los valores y la gráfica y otorga funcionalidades como ampliarla o descargarla.
Postcondición	Al detener la cámara, la gráfica desaparece respetando así el estado inicial de la interfaz
Excepciones	si no accede al sensor: N/A
Importancia	Baja

Tabla B.10: CU-10 Visualizar métricas del sistema.

CU-11	Limpieza de datos temporales
Versión	1.0
Autor	Hugo Gómez Martín
Requisitos asociados	RF-6
Descripción	Elimina los archivos subidos por el usuario tras usarlos
Precondición	Existen imágenes en el fichero temporal
Acciones	1. El usuario abandona Foto o termina Entrenamiento. 2. El sistema ejecuta un borrado del directorio temporal
Postcondición	Espacio liberado, no quedan datos personales
Excepciones	-
Importancia	Baja

Tabla B.11: CU-11 Limpieza de datos temporales.

Apéndice C

Especificación de diseño

C.1. Introducción

En este capítulo se documenta como está construido nuestro proyecto: los datos que maneja, el flujo interno y la arquitectura *software* que implementa.

C.2. Diseño de datos

Nuestra aplicación en ningún momento retiene información del usuario como *login* o *register* ni guarda absolutamente ningún dato personal debido a que no implementamos ni Base de Datos ni establecemos roles de usuarios, todo se genera o carga en memoria cada vez que el usuario la ejecuta.

Es cierto que el usuario puede añadir imágenes tanto en el entrenamiento, como en la clasificación vía imagen estática o en la detección en tiempo real, pero una vez realizada su función se libera de la memoria y no se almacena, detalladamente, esas imágenes subidas se copian a `TMP_PATH (/tmp/train_v3_tmp)`, un directorio temporal que se crea en cada sesión, después, pasa por un contenedor intermedio llamado *uploads* el cual crea un diccionario en memoria que agrupa los ficheros por clase.

Podemos abordar los datos de las estadísticas que se dibujan sobre la marcha (FPS, uso de CPU, RAM y temperatura) se conservan en `METRICS_BUFFER`, una *deque* de 120 posiciones que guarda las 120 lecturas mas recientes para dibujar el gráfico en tiempo real.

Por último, todos los parámetros globales o rutas a conjuntos de datos y modelos residen en el módulo *config.py*, lo que permite ajustar datos y parámetros de la aplicación modificando un solo archivo.

C.3. Diseño procedimental

Se van a implementar los siguientes diagramas de flujo que corresponden las tres casos de uso principales que se pueden realizar en nuestra aplicación:

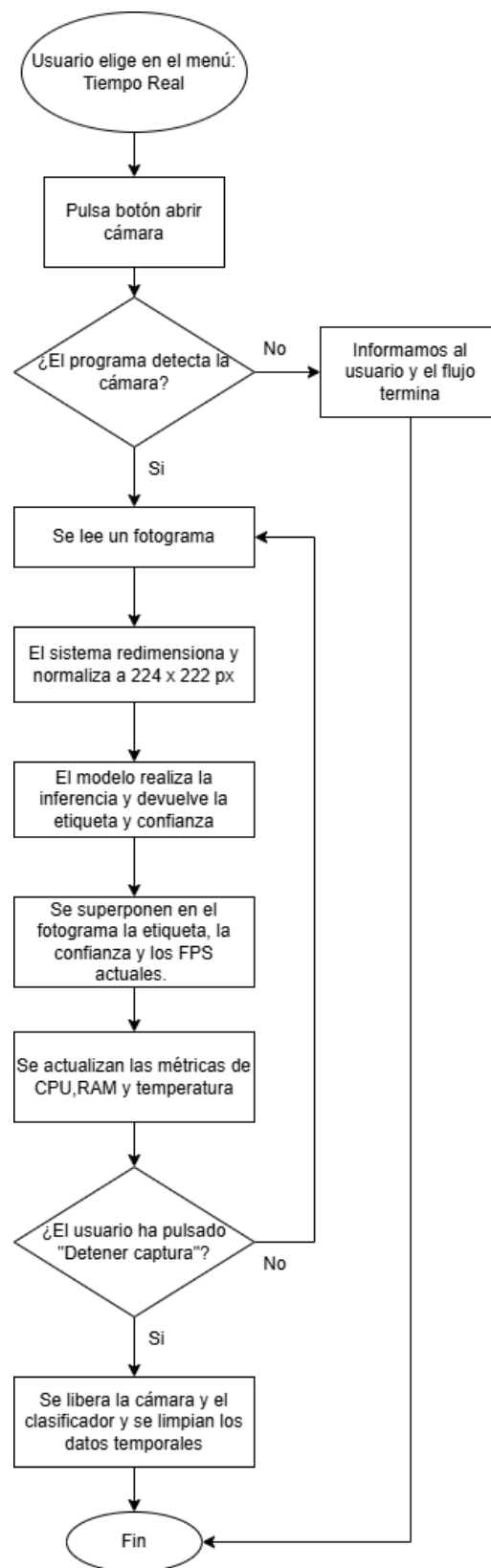


Figura C.1: Diagrama de flujo de Detección en tiempo Real

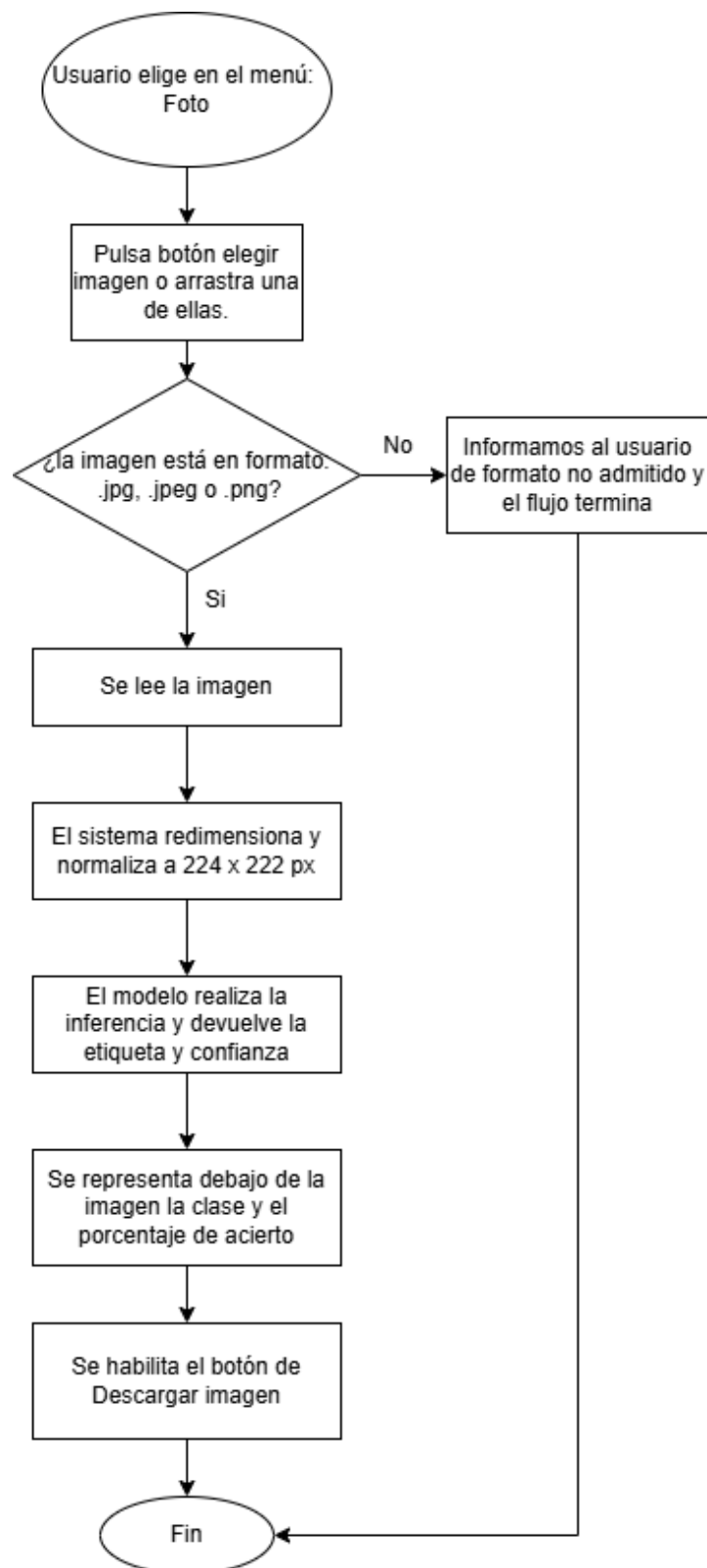


Figura C.2: Diagrama de flujo de Clasificación de imagen estática

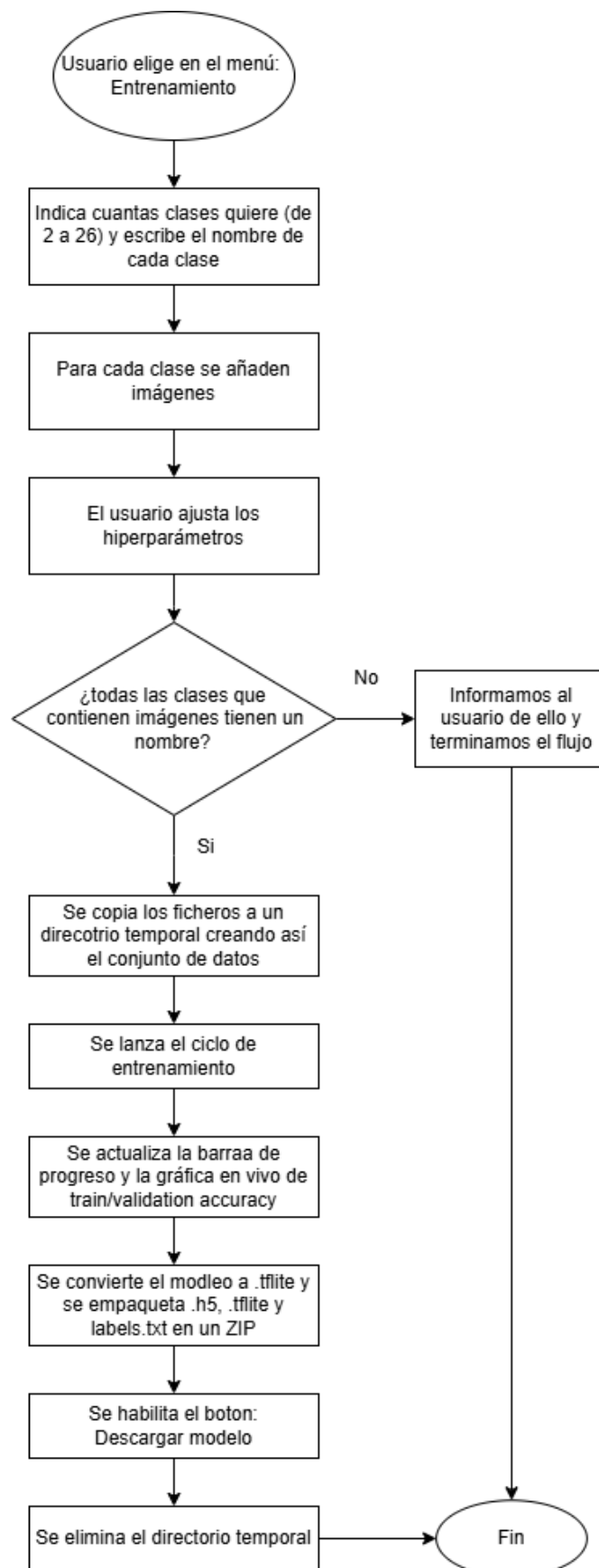


Figura C.3: Diagrama de flujo de Entrenamientos de modelos personalizados

Estos son solo 3 diagramas de flujo de los 11 casos de uso establecidos. Se pueden establecer mas pero es cierto que estos tres casos de uso (los mas importantes) abarcan y dan a entender y resumen bien el diseño procedimental de nuestra aplicación.

C.4. Diseño arquitectónico

En el proyecto a la hora de diseñar la arquitectura de nuestra aplicación en `Streamlit`, decidimos añadir un MVC (Modelo-Vista-Controlador) **ligero** adaptado al contexto de esta librería. `Streamlit` ejecuta el *frontend* y el *backend* en el mismo proceso, por lo tanto, la separación no es física (dos servicios), si no lógica. Aun así, distinguir estas capas nos ayuda a mantener el código modular y fácil de mantener.

Modelo-Vista-Controlador

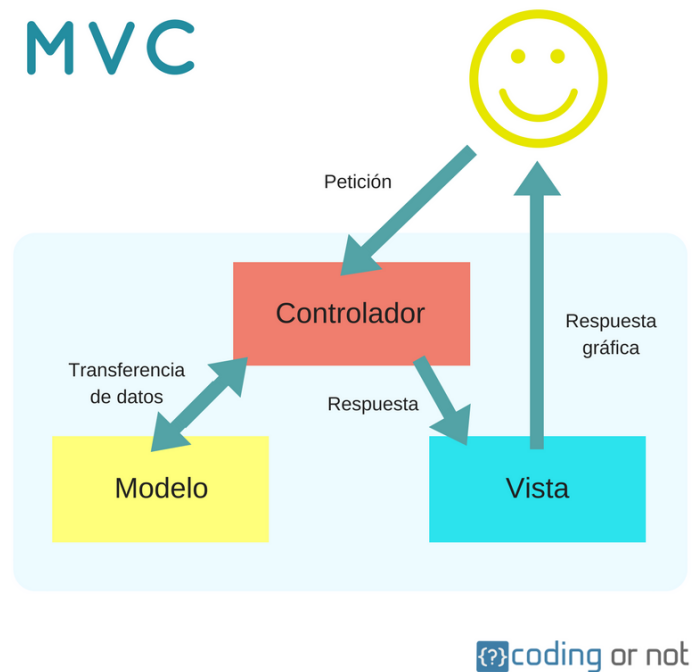


Figura C.4: Esquema de la arquitectura MVC [3]

En este capítulo, se establecerá la explicación lógica de la implementación de la arquitectura MVC en nuestro programa.

Modelo

El modelo agrupa toda la lógica que "sabe de IA" de tratamiento de imágenes:

- **Paquete `classifier/`:** Clases como `RealTimeASLClassifier` o `StaticPhotoClassifier` se encargan de clasificar las imágenes, preprocesar cada fotograma o fotografía además de redimensionar las imágenes y por último devolver tanto la etiqueta como la confianza de la clasificación.
- **Paquete `model_creation/`:** Establecido para que tiempo real en nuestra interfaz pueda usar la arquitectura del modelo principal para realizar el entrenamiento gracias a la importación de la clase `build_cnn_model`

Vista

La vista abarca todo lo que el usuario ve y toca que, en nuestro caso esta establecido por:

- Tiempo Real
- Foto
- Entrenamiento

Cada uno construye su respectiva interfaz con `Streamlit` con distintas funcionalidades (como por ejemplo, `st.button`, `st.file_uploader`, etc.) y presenta los datos que el modelo va produciendo en la interfaz de usuario.

Además el *frontend* de nuestra aplicación también cuenta con un CSS (`style.css`) el cual oculta el *footer*, define el rojo característico de Raspberry como color primario y ajusta la barra inferior. El resto de la paleta queda en manos de la configuración establecida en la carpeta `.streamlit` de la raíz.

Es importante mencionar que la Vista no conoce los detalles de *Tensorflow* ni de *OpenCV*, simplemente recibe el resultado procesado y lo enseña.

Controlador

El Controlador constituye la capa de coordinación entre el Modelo y la Vista. Es el encargado de dirigir el flujo de eventos y traducir las acciones de la interfaz de usuario en invocaciones al Modelo.

En nuestro proyecto se establece en `app.py` donde:

- Presenta el menú lateral (Tiempo real, Foto y Entrenamiento) e invoca a cada componente/interfaz cuando procede.
- Mantiene el estado de la sesión `st.session_state` para saber, por ejemplo, si la cámara está encendida o que modelo cuantizado está seleccionado.

Estilo de la aplicación

En este capítulo se resumirá la guía visual de la interfaz de usuario implementada en `Streamlit`.

Nombre

Antes de adaptar los colores y el estilo de la aplicación, tenemos que establecer previamente un nombre y un logotipo.

Como nombre se eligió “**ASL Detection**” . Como se puede observar, es muy simple y fácil de recordar. Además describe exactamente el objetivo del proyecto. Es cierto que también se barajó el nombre de “HandSign Detection” al igual que el nombre del repositorio (nombre que se le otorgó inicialmente al proyecto), pero era demasiado general y se decidió sustituir por el lenguaje de signos específico, que es el lenguaje de signos americano (ASL).

Logotipo de la aplicación

Al igual que con el nombre, establecimos un logo minimalista fácil de memorizar.

Se tuvieron en cuenta varias ideas, pero la que más destacó fue: establecer las siglas ASL en perfecta sincronía con bordes tanto redondeados como puntiagudos, además de darle un toque personal añadiendo el color oficial corporativo de Raspberry y dándole profundidad [2]:



Figura C.5: Logotipo de la aplicación ASL Detection

Colores

Una vez tenemos el Logotipo y el Nombre de la aplicación, podemos establecer los colores temáticos de la aplicación.

Los colores van a ir de la mano con el logotipo estableciendo 4 colores distintos:

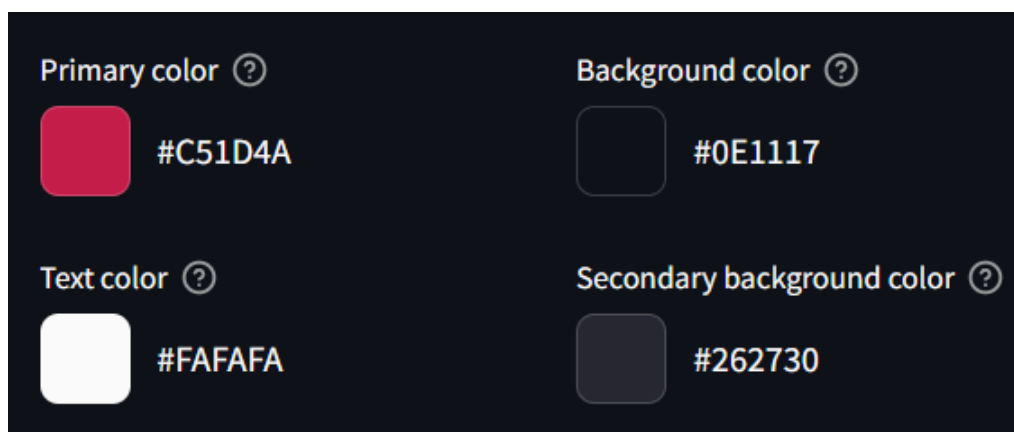


Figura C.6: Paleta de colores implementada en la aplicación

- **Color primario:** Se escogió este color debido a que es el mismo color que el del logotipo, es decir, cuenta con el mismo color que el color corporativo de Raspberry Pi.
- **Color del texto:** Color predeterminado para el texto y contrastarse mejor con un fondo negro.

- **Color de fondo:** fondo negro para destacar botones, títulos, gráficas, etc.
- **Color secundario del fondo:** fondo secundario oscuro implementado en la barra lateral para que el menú y el desplegable de elección del modelo puedan diferenciarse del fondo

todos estos colores están establecidos en el fichero `config.toml` que usa **Streamlit** para cargar distintas configuraciones, desde colores hasta dirección del puerto, etc.

Tipografía

La tipografía que se estableció fue la tipografía predeterminada de **Streamlit**: Sans Serif.

Es una tipografía clara y legible, por lo que no se optó por cambiarla

Inconografía

La interfaz como observamos a lo largo de la aplicación, recurre a *Emojis Unicode*:

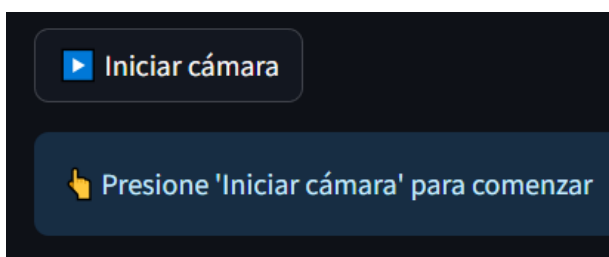


Figura C.7: Captura de la interfaz tiempo real

Son muy comunes en interfaces realizadas con **Streamlit** además de poder usarse en cualquier SO sin empaquetar fuentes, además refuerzan el sentido de la acción al primer golpe de vista para mejorar la comprensión del usuario.

Apéndice D

Documentación técnica de programación

D.1. Introducción

En este capítulo se detallarán aspectos relacionados con la documentación técnica, estructura de directorios, manual del programador, y distintos requisitos para la correcta ejecución del proyecto.

D.2. Estructura de directorios

El código reside en la carpeta `/src` donde las distintas funcionalidades están separadas por carpetas dependiendo de la función que relacen, la estructura de los directorios es:

- **`/src/hand_sing_detection/classifier`:** es el motor de inferencia de nuestra aplicación que contiene 3 clasificadores distintos, uno orientado a la clasificación en tiempo real en `classifier_rpy.py` y la inferencia en `classifier_photo.py` además un fichero `classifier.py` que realiza la inferencia únicamente de modelo sin optimizar donde podemos probar nuestro modelo (orientado para probar modelos por parte del programador no accesible desde la aplicación). Estos modleos cargan modleos *TensorFlow/Keras*, hace el pre-procesado y devuelven la etiqueta ASL con su respectiva confianza, ya sea fotograma a fotograma (tiempo real) o sobre una imagen estática

- **/src/hand_sing_detection/dataset:** Este directorio permite crear el *dataset*. el fichero `collection.py` recoge unicamente imágenes 300x300 pixel procesadas donde se irán almacenando en la carpeta */data/ordered*. Te permite crear mas subdirectorios (clases) y almacenar el conjunto de datos estableciendo un limite. El fichero `disordered.py` es el encargado de desordenar los datos para que a la hora de entrenar el modelo cada lote durante el entrenamiento contiene muestras mixtas de los dos conjuntos de datos mezclados, lo que favorece la generalización del modelo.
- **/src/hand_sing_detection/model_creation:** Contiene dos ficheros: `cnn_build.py` que se encarga de entrenar el modelo con los hiperparámetros otorgados por el *Random search* añadiendo *Callbacks* como *Early stopping* o *ReduceOnPlateau*. Su función `build_cnn_model()` es la que contiene la arquitectura del modelo y la usa la interfaz de entrenamiento de la aplicación. El fichero `random_search.py` cuenta con la misma arquitectura pero con la diferencia de que contiene **Keras tuner** para la búsqueda de hiperparámetros.
- **/src/hand_sing_detection/utils:** en este directorio podemos encontrar el fichero que convierte el modelo exportado *.h5* en dos ficheros *.tflite* cuantizados (FP32 y FP16).
- **/src/hand_sing_detection/web** En este directorio podemos encontrar `app.py` el cual gestiona las demás interfaces además de proporcionar la interfaz “Inicio” y proporcionar la barra lateral con el menú y desplegable del modelo a usar.
 - **/src/hand_sing_detection/web/components:** Se proporcionan 3 Ficheros para clasificación en tiempo real, clasificación estática por foto y entrenamiento.
 - **/src/hand_sing_detection/web/static:** Contiene `style.css` el cual aplica el footer de nuestra app y le proporciona el mismo color principal que al resto de la web además de contener la carpeta **images** que es donde se ubica el logotipo empleado en la pagina web y en la pestaña

ademas, a la misma altura que estas carpetas en encuentra `config.py` que contiene todas las rutas relativas necesarias para los ficheros así como sus variables globales.

Además, fuera del *src/* hay mas ficheros que no contienen código pero sí cosas importantes a destacar que son esenciales para el funcionamiento de nuestro proyecto.

- **models/**: Contiene los modleos cuantizados y normales junto a las respectivas labels.txt del modelo principal, del *Random Search* y de *Teachable machine*.
- **data/**: contiene los conjuntos de datos */ordered* y */disordered* con 26000 imágenes repartidas en 26 clases cada uno.
- **/.streamlit/config.toml**: contiene el fichero de configuración de *Streallmit* que contiene los colores de la aplicación junto a el puerto y el máximo de subida de la aplicación.
- También hay ficheros sueltos como **run_app.sh/run_app.bat** que son lanzadores rápidos para Linux y Windows los cuales instalan dependencias y ejecutan **app.py** para iniciar la aplicación en nuestro navegador además de contener **ASL_Detection.exe** el cual lanza la aplicación con el único requisito que es tener Python 3.11 instalado. Este ejecutable se encuentra en el apartado de Releases en nuestro repositorio.

D.3. Manual del programador

En esta sección vamos a explicar como crear el entrono de desarrollo para poder trabajar con el código:

Clonar el repositorio

Lo primero de todo sería clonar el repositorio para poder tener acceso local a todas las carpetas, modelos, conjuntos de datos, requerimientos, etc. que necesitamos para trabajar con el proyecto.

Para poder realizar eso, necesitamos copiar la dirección web y bajarlo al directorio de nuestra preferencia, una vez nos encontremos en el, abrimos la terminal del sistema y ejecutamos el siguiente comando:

```
git clone https://github.com/hgomezmartin/HandSignDetectionProject.git
```

una vez clonado, podremos abrirlo en nuestro IDE favorito, personalmente recomiendo PyCharm debido a la cantidad de funcionalidades que tiene

por defecto, como crear un entorno virtual automáticamente o instalar requerimientos de forma sencilla.

Instalación de Python

La versión de Python que ha sido utilizada en este proyecto ha sido la versión 3.11.7 específicamente, por lo tanto se recomienda utilizar esa misma versión o una versión compatible con el proyecto.

Windows

Para poder instalarlo en Windows, debemos descargar desde la página web oficial de Python la versión [3.11.7](#).

Una vez seguidos todo estos pasos de instalación y haberlo agregado al *Path* del sistema, comprobamos mediante este comando en la consola del sistema la instalación de Python:

```
python -version
```

la consola nos debería mostrar la versión de Python instalada.

Linux

Para poder instalar Python, se tendrá que realizar a través de terminal:

```
sudo apt install python3.11
```

una vez instalado, verificamos por terminal que esté correctamente instalado para su uso.

```
python3.11 -version
```

Instalación de dependencias

Recomendamos activar un entorno virtual para separar estas dependencias, pero el cada usuario es libre de poder elegir donde instalarlo.

Para poder crear el entorno virtual y activarlo, necesitaremos estos dos comandos

En Windows (si no quieres instalarlo en un entorno virtual, ignora los dos primeros comandos):

```
python -m venv .(nombre (recomendamos .venv_handsign)) (crea el entorno)
```

```
./.(nombre del entorno)/Scripts/activate (lo activa)
```

una vez activado, instalaríamos las dependencias con el comando:

```
pip install -r requirements.txt
```

En linux:

```
python3 -m venv .(nombre (recomendamos .venv_handsign)) (crea el entorno)
```

```
source .(nombre del entorno)/bin/activate (lo activa)
```

y después se instalarían las dependencias al igual que en Windows desde pip:

```
pip install -r requirements.txt
```

D.4. Compilación, instalación y ejecución del proyecto

A continuación, después de haber instalado los requerimientos básicos del proyecto, se va a establecer el camino recomendado para dejar el proyecto operativo desde cero.

Instalar el paquete del proyecto en modo editable

El primer paso consiste en registrar e propio código como paquete Python para que la aplicación pueda importar sin errores los módulos de entrenamiento y clasificación (en la raíz del proyecto):

```
python -m pip install -e .
```

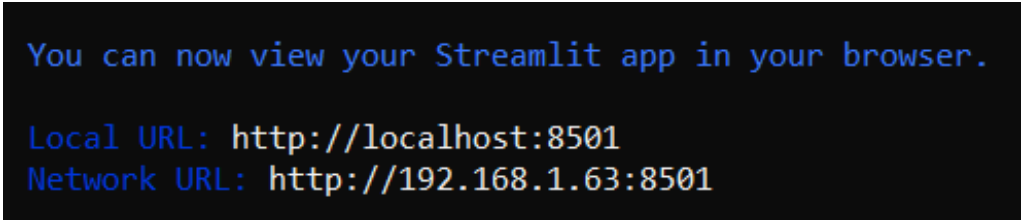
Levantar la aplicación manualmente

Desde la raíz del proyecto, se procede a la ejecución de la aplicación:

```
streamlit run src/handsign_asl_detection/web/app.py
```

No obstante, hay ficheros preparados en la raíz del proyecto que instalan dependencias, crean el paquete de la aplicación y abren automáticamente la aplicación sin necesidad de estos comandos, la única dependencia que necesitan es tener Python instalado. Además también se incluye un ejecutable (.exe) aunque es cierto que debemos tener Python 3.11 o superior instalado en nuestro equipo para poder ejecutarlo.

Una vez **Streamlit** se ejecute, la consola mostrará algo como:

A screenshot of a terminal window with a black background and light blue text. The text displayed is: "You can now view your Streamlit app in your browser." followed by "Local URL: http://localhost:8501" and "Network URL: http://192.168.1.63:8501".

```
You can now view your Streamlit app in your browser.  
Local URL: http://localhost:8501  
Network URL: http://192.168.1.63:8501
```

Figura D.1: Captura de las URL que nos muestra Streamlit por consola

Una vez cargue completamente **Streamlit** y aparezca eso, se nos abrirá automáticamente en el Navegador la aplicación. No obstante, si queremos acceder desde algún otro dispositivo que este conectado a la misma red, insertaríamos la *Network URL* y accederíamos a la aplicación.

Para desconectarla sería, cerrar la sesión con “ctrl+c” en la terminal o cerrar directamente la consola.

D.5. Pruebas del sistema

Es cierto que no se han implementado baterías de test automatizados. No obstante, antes de considerar la aplicación lista, se comprobó el funcionamiento previo de todos los códigos para garantizar su correcto funcionamiento y la cobertura funcional completa de los distintos casos de uso definidos.

Apéndice E

Documentación de usuario

E.1. Introducción

En este capítulo se detallarán aspectos relacionados con el correcto funcionamiento de la aplicación, siendo explicada paso a paso con el objetivo de que un usuario inexperto pueda ejecutar y manejar la interfaz de usuario correctamente conociendo todas las funcionalidades permitidas por la aplicación.

E.2. Requisitos de usuarios

Estos son los requisitos que debe cumplir el usuario para poder ejecutar nuestra aplicación web.

- Un ordenador que cuente con un sistema operativo:
 - Windows 10 o superior
 - Ubuntu 22.04 o superior
 - Raspberry Pi OS Bookworm

En resumen, cualquier sistema operativo de 2021 en adelante con 64 bits cumple, versiones anteriores no garantizan compatibilidad con Python 3.11.

- Una cámara conectada al dispositivo, la cual no es necesaria si no vas a usar la inferencia en tiempo real.

- Al menos 1GB libre de almacenamiento para guardar los ficheros temporales (modelos + imágenes)
- Un navegador que tenga JavaScript habilitado y soporte a Websockets. la mayoría de los mas conocidos como Opera, Google Chrome, Firefox, Edge lo tienen implementado.

E.3. Instalación

Para su respectiva información, hay varias formas de poder ejecutarlo.

Ejecutar ejecutable

Si estamos en Windows, la foma mas sencilla de poder ejecutar nuestra aplicación una vez Python 3.11 o superior esté instalado en nuestros equipos, es ejecutar el ejecutable `.exe` del apartado de *releases* en nuestro repositorio.

No obstante, si estamos en Linux o la versión ejecutable en Windows nos da problemas, solo nos hará falta bajarnos la version 3.11 de Python:

- **Microsoft Windows:** Ir a la página oficial de Python e instalar la versión compatible [3.11.7](#).
- **Pi OS/ Linux:** Instalar con el siguiente comando en consola: `sudo apt install python3.11`

Una vez instalado correctamente, en L  inux correr  amos como programa `run_app.sh` y esperar  amos a que se bajasen todas las dependencias para que la aplicaci  n se inicie autom  ticamente en nuestro navegador.

En Windows hacemos doble clic en `run_app.bat` y esperamos, al igual que en Linux, a que se bajen todas las dependencias, despu  s se ejecutar   autom  ticamente en nuestro navegador.

Si no se llegase a ejecutar autom  ticamente, copiar  mos la *Local URL* que aparece en la terminal.

E.4. Manual del usuario

Una vez ejecutado, procedemos a explicar todos los apartados y funcionalidades de la aplicaci  n para que el usuario pueda manejarse de forma

fluida por la interfaz. Todas las explicaciones irán acompañadas por ciertas capturas para que el usuario pueda asociar las explicaciones con la interfaz.

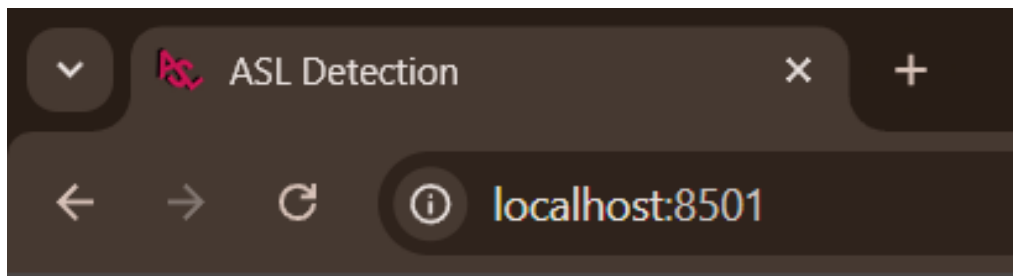


Figura E.1: Captura de la pestaña de nuestra web con logo y título

Recordemos que nuestra aplicación tiene 3 funcionalidades principales, Detectar lenguaje de signos ASL en tiempo real, Clasificación mediante foto estática y Entrenamiento con nuestro propio conjunto de datos. Una vez reordenado todo esto, procedemos a la explicación de toda la interfaz incluyendo su funcionalidad.

Página de inicio

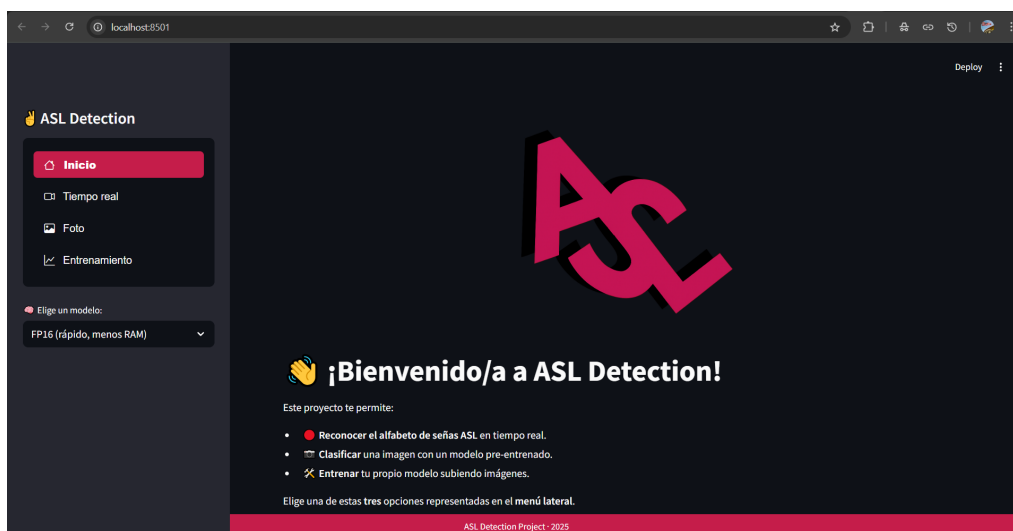


Figura E.2: Captura de la interfaz de inicio de la aplicación

Principalmente nos encontraríamos con esta interfaz donde se muestra la página de inicio y se describe todo lo que la aplicación es capaz de realizar:

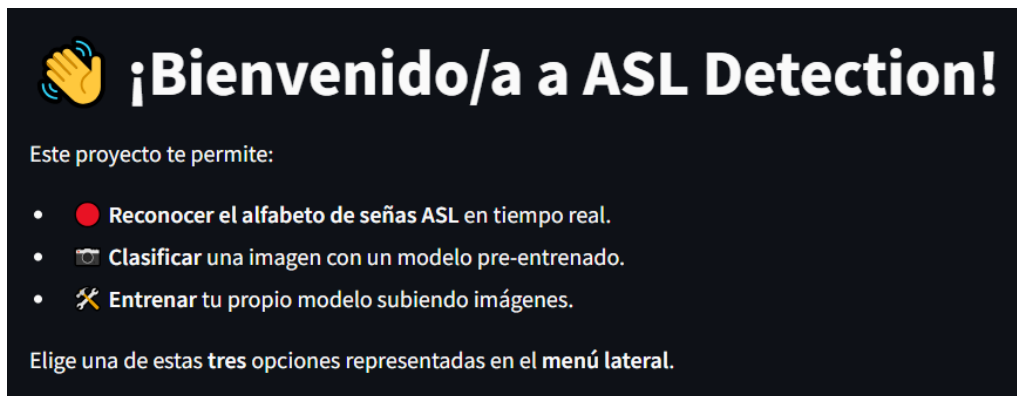


Figura E.3: Captura ampliada de la interfaz de inicio para apreciar mejor la descripción

Barra lateral

La barra lateral como se puede apreciar esta inicialmente desplegada (configurada inicialmente así), si en algún momento nos puede llegar a molestar, se puede esconder gracias a la flecha ubicada en la esquina superior derecha.

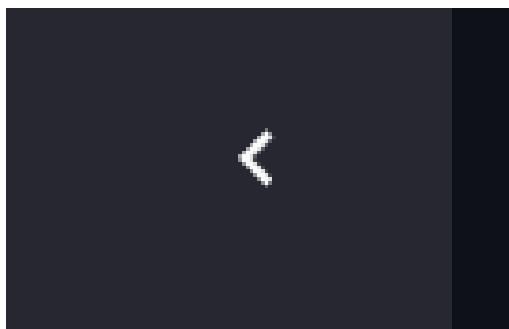


Figura E.4: Captura de la flecha establecida en la barra lateral

Menú

Gracias a la implementación de este menú, el usuario podrá navegar por la interfaz de la manera mas intuitiva posible. También se establecieron

nombres acortados y símbolos para que el usuario detecte rápidamente la funcionalidad de cada opción del menú

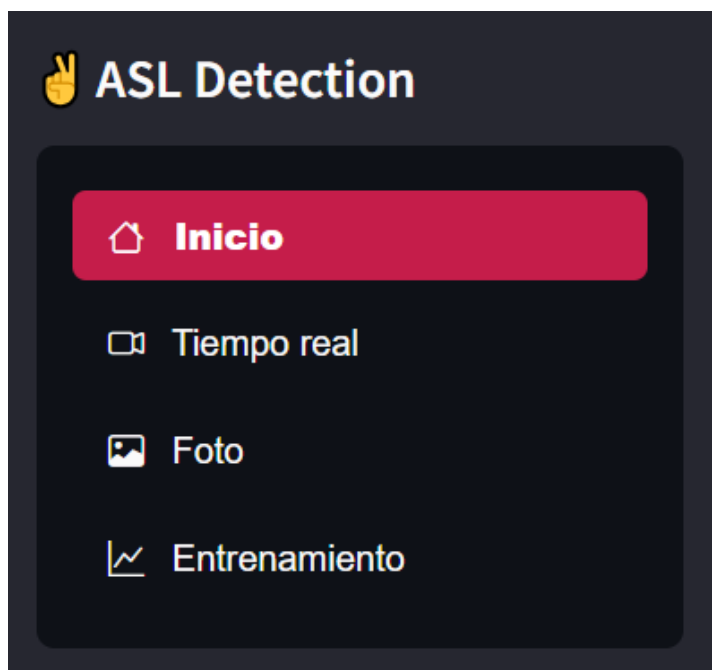


Figura E.5: Captura del menú presente en la barra lateral

Desplegable Modelo

Para la inferencia se aportaron dos modelos que se aplicarán a todas las funcionalidades de la página web, los cuales podemos elegirlos desde un desplegable justo debajo de el menú principal integrado en la barra lateral

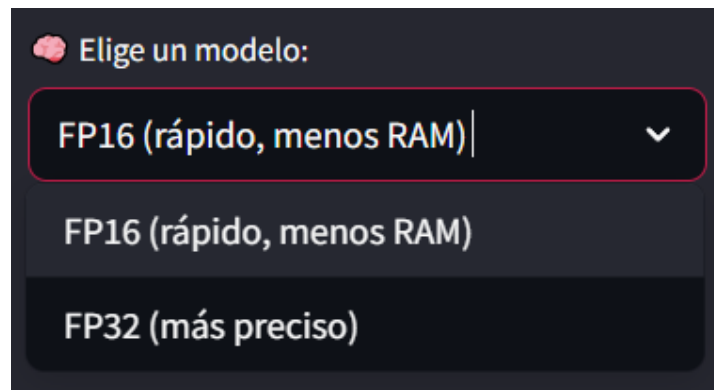


Figura E.6: Captura de la interfaz para elegir un modelo (FP32 o FP16)

Reconocimiento en tiempo real

En esta interfaz se muestra al usuario el título confirmando que está en la interfaz correcta además de implementar un botón para empezar la inferencia y una pequeña descripción para animar al usuario a pulsar el botón.

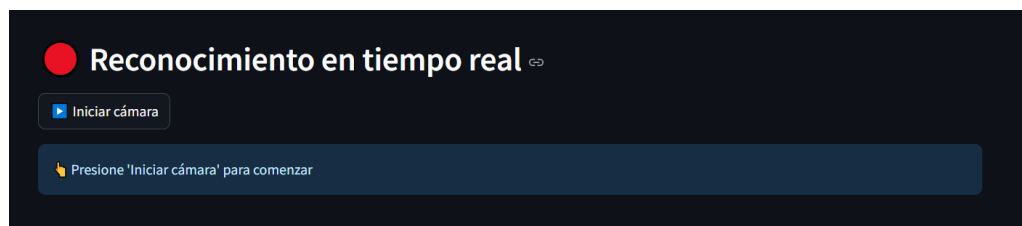


Figura E.7: Captura de la interfaz inicial de Tiempo Real

Pulsamos iniciar cámara y esta es la respuesta de la web.

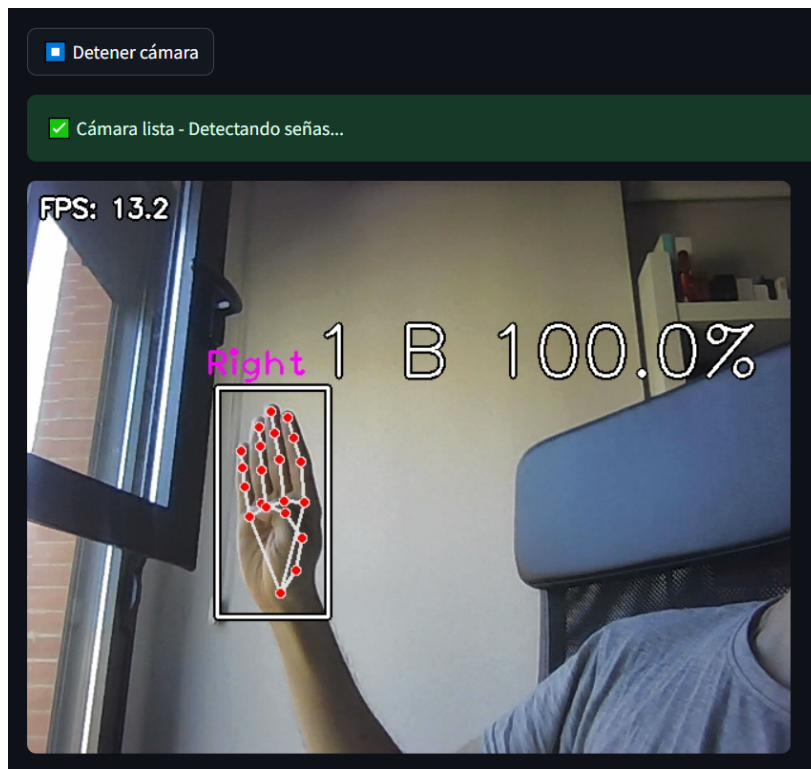


Figura E.8: Captura de la parte de inferencia de la interfaz Tiempo Real

Como podemos observar en la imagen de arriba, podemos observar el botón de Detener cámara para detener la captura y volver al estado inicial. También se puede apreciar que la descripción cambió confirmando así que la cámara está lista para detectar señas. Además también se puede apreciar los FPS en la esquina superior izquierda de la imagen y nuestra mano rodeada por el *bounding box* donde podemos apreciar la mano detectada (automáticamente establecida por la biblioteca *cvzone*), el número de la clase acertada, en este caso el 1 referente a la letra B, la letra acertada, que en este caso es la letra B y el porcentaje de confianza.

Justo abajo tenemos todas las métricas y un gráfico que se va actualizando cada dos segundos.

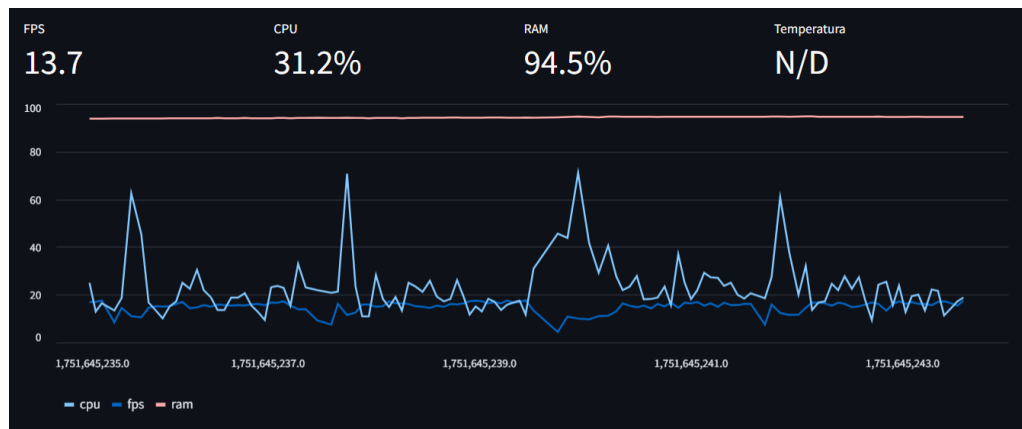


Figura E.9: Captura de la parte de métricas de la interfaz tiempo real

Como podemos observar, la temperatura no esta disponible en Windows, pero en la Raspberry sí, por eso sigue implementado la temperatura en las métricas.

Reconocimiento por foto

En esta interfaz, el objetivo es clasificar una imagen estática de nuestra mano que sea *.jpg*, *.jpeg* o *.png*

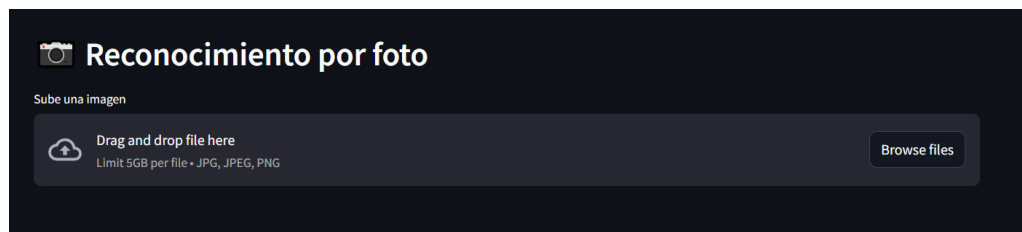


Figura E.10: Captura de la interfaz inicial de Foto

Si intentamos subir una imagen no compatible saltará el siguiente aviso:

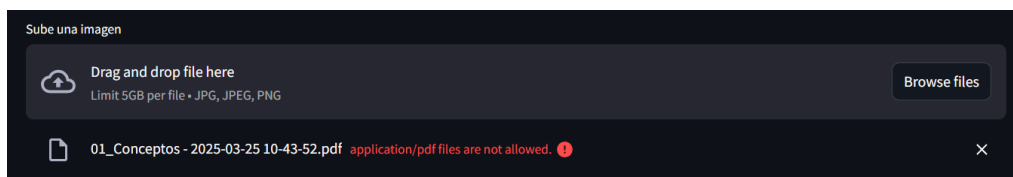


Figura E.11: Captura de la interfaz inicial foto que no acepta el fichero por incompatibilidad

una vez subido una foto compatible con el formato requerido, muestra la predicción y el formato descarga por si necesitas descargarla de nuevo.

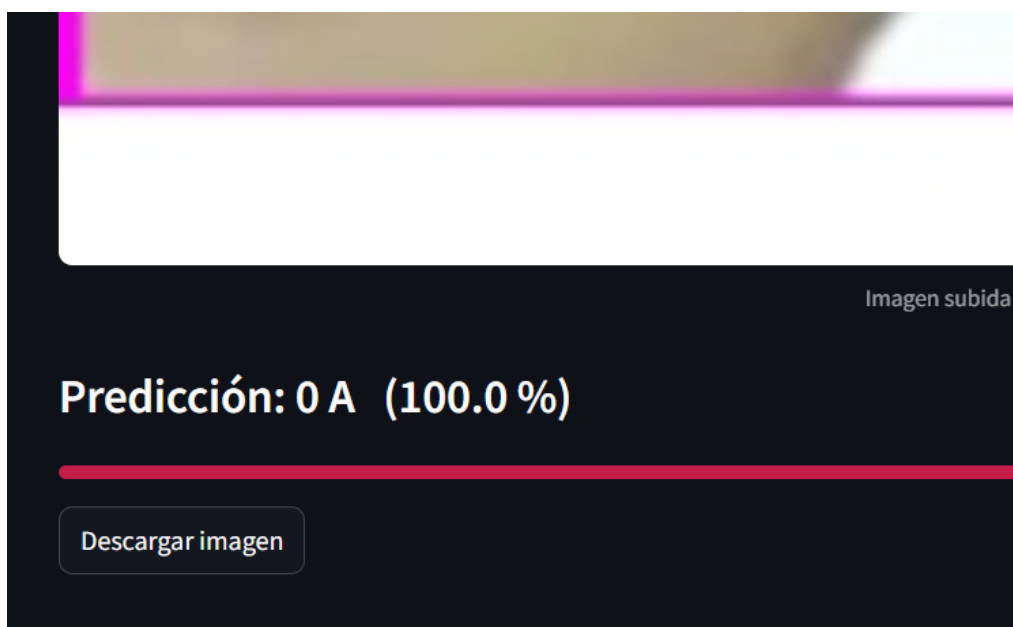


Figura E.12: Captura de la interfaz foto una vez realizada la inferencia

Entrenamiento

En esta interfaz se puede entrenar con nuestro propio conjunto de datos las clases cuales queramos hasta un límite de 26 clases totales además de poder modificar los tres hiperparámetros a través de *sliders*.



Figura E.13: Captura de la interfaz de entrenamiento inicial

Por lo tanto, así quedaría una clase después de añadir 1000 imágenes al igual que en nuestro conjunto de datos:

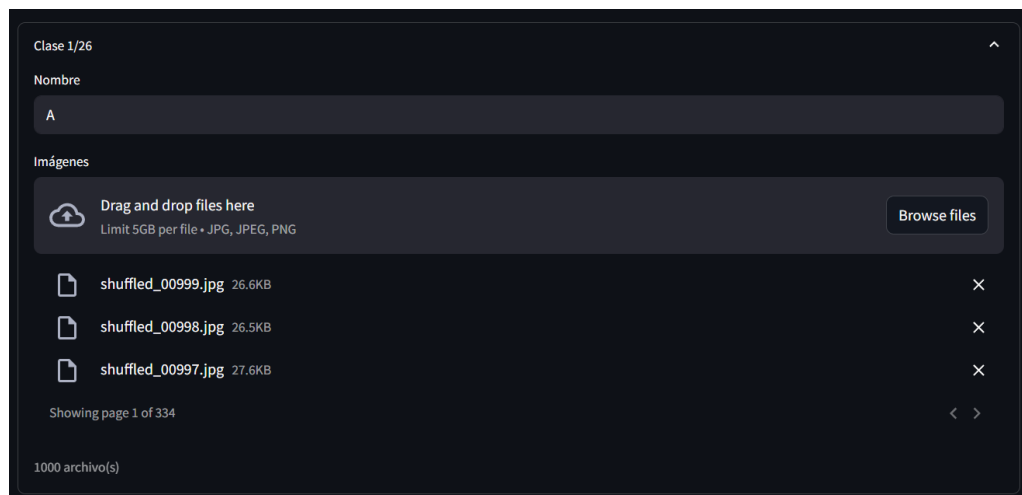


Figura E.14: Captura de la interfaz entrenamiento añadiendo 1000 imágenes

Vamos a iniciar el entrenamiento (voy a mostrar un entrenamiento *dummy* debido a que estoy con el portátil y clasificar tantas imágenes

tardaría bastante, por lo que se han elegido dos clases (perro y gato) con 8 imágenes cada una) solo para demostración.

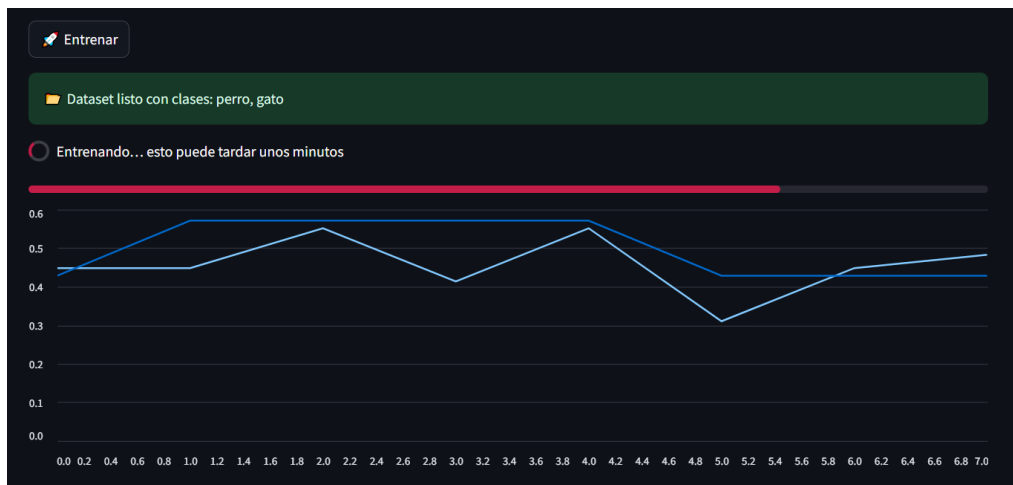


Figura E.15: Captura de la interfaz de entrenamiento en el proceso de entrenamiento

Una vez finalizada, podemos descargar el fichero comprimido `.zip` el cual contiene el modelo normal, optimizado y el fichero de texto con las etiquetas.



Figura E.16: Captura de la interfaz de entrenamiento finalizada

↑ hand_sign_model_1751648099.zip - archivo ZIP, tamaño descomprimido 32.775.267 bytes

Nombre	Tamaño	Comprimido	Tipo	Modificado	CRC32
..			Carpeta de archivos		
labels.txt	11	13	Documento de texto	04/07/2025 18:...	2ADE030C
model.h5	16.723.648	15.086.492	Archivo H5	04/07/2025 18:...	A7FAC396
model_fp32.tflite	16.051.608	14.853.156	Archivo TFLITE	04/07/2025 18:...	6D3A4279

Figura E.17: Captura del fichero .zip descomprimido

Anexo de sostenibilización curricular

F.1. Introducción

Este capítulo se incluye una reflexión personal sobre los distintos aspectos de la sostenibilidad que se abordan en este proyecto. A continuación expongo mi reflexión sobre los principios de sostenibilidad aplicados durante la realización de este Trabajo Fin de Grado, siguiendo las directrices de la [CRUE](#).

F.2. Competencias de sostenibilidad

Pensamiento enfocado a la sostenibilidad

Al planificar el proyecto analizamos múltiples factores a considerar y realizamos una cadena completa: dispositivo de captura, equipos con *hardware* limitado, interfaz de usuario, colectivo beneficiario y mantenimiento futuro. Ese mapa de me permitió priorizar cambios que aportaban más valor social sin aumentar la huella ambiental.

Responsabilidad social y accesibilidad

Diseñé la interfaz con contraste alto de colores, navegación por teclado y mensajes cortos sencillos para facilitar a los usuarios su navegación por la aplicación web de reconocimiento ASL. Además, todos los procesos se

ejecutan localmente, no se guardan imágenes ni se envían datos a terceros, lo que protege la privacidad de los usuarios.

Accesibilidad del proyecto

Siempre hemos tenido en cuenta a la hora de realizar nuestro proyecto una aplicación ligera y accesible para todos. Desde un primer momento se quiso resaltar la idea de que dispositivos computacionalmente muy limitados y pequeños actualmente son capaces de correr modelos para realizar clasificación de imágenes con modelos optimizados y cuantizados, Demostrando que la clasificación visual no está limitada únicamente a equipos de alto rendimiento.

Uso responsable de recursos y software libre

Para promover la reutilización, todo el proyecto se ha desarrollado con dependencias *open-source* apoyándonos en librerías con licencias permisivas (MIT, Apache 2.0, BSD). este enfoque evita costes de licencias y facilita el uso de estas en nuestro proyecto. Además, minimizar la instalación de software adicional y reutilizar módulos existentes contribuye a un menor consumo de recursos y favorece un ciclo de desarrollo más ágil, colaborativo y sostenible.

F.3. Conclusión

La elaboración de este proyecto sobre la detección de gestos de lenguaje ASL me ha demostrado que la sostenibilidad no es un extra, sino algo que se puede (y se debe) tener presente desde la primera línea de código.

Al optimizar el modelo para que funcione en una Raspberry Pi y al utilizar librerías de código abierto además de cuidar la accesibilidad de la interfaz, he visto cómo decisiones pequeñas marcan la diferencia: mayor rendimiento en sistemas con bajos recursos, menos barreras para quien quiera usar el sistema y más facilidad para implementar actualizaciones en un futuro.

Sobre esto último, a corto plazo quiero realizar modificaciones presentadas como líneas a futuro en la memoria. Así cierro un proyecto que, además de reconocer lenguaje ASL, demuestra que se puede hacer IA de forma eficiente, abierta y útil para todos.

Bibliografía

- [1] 4GeeksAcademy. ¿cuánto gana un programador en españa?, 2025. URL <https://4geeksacademy.com/es/cuanto-gana-un-programador/cuanto-gana-un-programador-en-espana>. [Internet; consultado 1-julio-2025].
- [2] Raspberry Pi Foundation. Colours, 2025. URL <https://bits.raspberrypi.org/#!/Colours>. [Internet; consultado 18-junio-2025].
- [3] Miriam García. Mvc. URL <https://codingornot.com/wp-content/uploads/2017/10/mvc-modelo-vista-controlador.png>.